

## Parte 3. Interpretación de comandos

---

Desarrollo de Aplicaciones y Servicios Virtuales - Primer Examen Parcial

### 1. Análisis de comandos

---

#### Planteamiento:

```
git status
git add README.md
git commit -m "Actualiza documentación"
git push
```

Explica qué ocurre en cada instrucción.

#### Respuesta y explicación:

- **git status** - Muestra el estado actual del repositorio: qué archivos fueron modificados, cuáles están en el área de preparación (staging) y cuáles no están siendo rastreados todavía. No modifica nada; solo informa.
- **git add README.md** - Mueve el archivo `README.md` al área de preparación (staging area). Esto le indica a Git “quiero que este archivo, con su estado actual, forme parte del próximo commit”. Los cambios todavía no quedan registrados de forma permanente en el historial.
- **git commit -m "Actualiza documentación"** - Crea una nueva versión (un commit) en el historial **local** del repositorio, guardando de forma permanente los cambios que estaban en el área de preparación, junto con un mensaje descriptivo que explica qué se hizo.
- **git push** - Envía el o los commits nuevos del repositorio local hacia el repositorio remoto (por ejemplo, GitHub), para que queden publicados y disponibles para otras personas o dispositivos.

En conjunto, esta secuencia representa el flujo estándar de Git: revisar el estado, seleccionar los cambios (stage), registrarlos en el historial (commit) y publicarlos en el remoto (push).

---

### 2. Identifica qué falta

---

#### Caso A

##### Planteamiento:

```
Modificar archivo
|
git add .
|
¿?
|
git push
```

Indica qué operación falta y explica su función.

**Respuesta:** Falta `git commit -m "mensaje descriptivo"`.

**Explicación:** `git add .` únicamente coloca los cambios en el área de preparación; todavía no existen como una versión guardada en el historial. Antes de poder enviarlos al repositorio remoto con `git push`, es obligatorio crear un commit, que es lo que efectivamente registra los cambios en el historial local de Git con un mensaje que describe qué se modificó. Sin ese commit no hay nada nuevo que `git push` pueda enviar.

#### Caso B

##### Planteamiento:

```
Repositorio GitHub
|
¿?
|
Repositorio local
```

Indica qué operación utilizarías y explica por qué.

**Respuesta:** `git clone <url-del-repositorio>` .

**Explicación:** Cuando todavía **no existe** una copia local del repositorio (es la primera vez que se va a trabajar con él en esa computadora), la operación adecuada es `git clone` . Esta descarga el repositorio completo desde GitHub -incluyendo todo su historial de commits- y crea la carpeta local del proyecto, ya conectada al repositorio remoto de origen.

## Caso C

**Planteamiento:**

```
Repositorio remoto actualizado
|
¿?
|
Repositorio local actualizado
```

Indica qué operación utilizarías y explica por qué.

**Respuesta:** `git pull` .

**Explicación:** A diferencia del Caso B, aquí ya **existe** una copia local del repositorio; lo que ocurrió es que el repositorio remoto recibió cambios nuevos (por ejemplo, de otro colaborador) que el repositorio local todavía no tiene. `git pull` trae esos cambios nuevos desde el remoto y los combina con el contenido local, dejando ambas copias sincronizadas. (Internamente `git pull` equivale a un `git fetch` que descarga los cambios, seguido de un `git merge` que los integra en la rama local.)